Site configuration

Website settings live in `calepin.toml` at the root of your website source directory. Use `--config <path>` only when the config file lives somewhere else.

Basic settings

A small site can start with:

```toml
# Site name shown by bundled themes and page metadata.
# Default: none.
title = "My Site"

# Short site description for metadata and previews.
# Default: none.
description = "A static website built from Typst documents."

# Public site URL, used for sitemap.xml and canonical URLs.
# Default: none.
base-url = "https://example.com"

# Theme: "calepin", "academic", a local theme directory, or false for raw output.
# Default: "calepin".
theme = "calepin"

# Render .pdf files for every page.
# Default: false.
pdf = false

# Publish each page's Typst source: copy .typ files to the output directory
# and embed the source for the theme's view-source mode.
# Default: true.
typ = false

# Minify generated HTML, including inline CSS and JavaScript.
# Default: false.
minify = true

# Default output directory for website builds, relative to calepin.toml.
# A positional output path on the command line takes precedence.
# Default: build in place, in the website source directory.
output-dir = "../_site"

# Static search engine. Set to "pagefind" to generate a search index.
# Default: none.
search = "pagefind"

# Generate web feeds (requires base-url). Use `feeds = true` for a default
# atom.xml, or a [feeds] table for custom filenames and templates.
# Default: false.
feeds = true

# Generate llms.txt, a Markdown index of the site for LLM consumers.
# Default: true.
llms = false

# Logo image path for the top bar.
# Default: none.
logo = "assets/logo.svg"

# Alternative text for the logo image. Defaults to `title` when omitted.
# Default: none.
logo-alt = "My Site"

# Output directory for browser-facing generated assets.
# Also controls where the Calepin runtime is written.
# Default: ".calepin".
asset-dir = "_calepin"
```

```toml
# Browser favicon path. Omit this to use Calepin's generated default.
# Default: `asset-dir`/favicon.svg.
favicon = "assets/favicon.ico"

# Default social-card image for link previews. Needs `base-url` unless the
# value is already an absolute URL. Pages override it with `image`.
# Default: none.
image = "assets/social-card.png"

# Tint applied to browser UI on mobile.
# Default: none.
theme-color = "#1b1b1b"

# HTML syntax highlighting themes. Paths are relative to calepin.toml.
# Default: built-in Calepin light and dark themes.
highlight-light = "themes/syntax/light.tmTheme"
highlight-dark = "themes/syntax/dark.tmTheme"
```

Multi-word keys use kebab-case (`base-url`, `logo-alt`, `output-dir`). The older snake_case spellings (`base_url`, `logo_alt`, ...) are still accepted for backward compatibility.

Use `--format html` for a one-time HTML-only build, regardless of `pdf`. Use `--minify` to minify HTML for a single build without changing `calepin.toml`.

By default every page ships its Typst source: the `.typ` file is copied next to the rendered HTML, and the full source is embedded in the page so the theme's view-mode picker can show it. Set `typ = false` to withhold both. Bundled themes list only the views that exist, so a site with `pdf = false` shows no PDF entry, one with `typ = false` shows no Source entry, and an HTML-only site drops the picker entirely.

Set `search = "pagefind"` to add static search to bundled website themes. *Calepin* writes the Pagefind search bundle to `pagefind/` after rendering the site and links the bundled themes to the generated component script and stylesheet. Bundled themes mark the main page content with `data-pagefind-body`, so navigation, toolbars, and footers are excluded from the search index.

Paths in `calepin.toml` are interpreted from the website source directory: the directory that contains the config file, unless you pass an explicit `--config`. This includes `logo`, `favicon`, page `target` values ending in `.typ`, page `glob` patterns, page and static `include` and `exclude` patterns, and local theme paths. During the build, *Calepin* rewrites internal files and generated assets as page-relative URLs, so links and images continue to work from nested pages. Literal URLs such as `https://example.com` are left unchanged.

Auto-generated metadata

Bundled website themes emit a small set of page-level head tags from config:

- `<title>` (fallback when no `<title>` is present in the rendered page)
- `<meta name="description">`, `<meta property="og:description">`, and `<meta name="twitter:description">` from the page's own summary or excerpt, falling back to the site `description`
- `<meta property="og:title">` and `<meta name="twitter:title">` from page title and site title
- `<meta property="og:site_name">` from `title`
- `<meta property="og:url">` and `<link rel="canonical">` from `base-url` plus the page route
- `<meta property="og:type">`: `article` for a page with its own title, `website` otherwise
- `<meta property="og:image">`, `<meta name="twitter:image">`, and `<meta name="twitter:card">` from `image`
- `<meta name="theme-color">` from `theme-color`

If you inject your own tags for these fields in a theme override, check for duplication with what the theme already emits.

Social cards

`image` sets the picture link previews show on social platforms and chat apps:

```
# calepin.toml

# Default social-card image. Relative paths resolve from the website source
# directory, exactly like `logo` and `favicon`.
# Default: none.
image = "assets/social-card.png"

# Tint for browser UI on mobile.
# Default: none.
theme-color = "#1b1b1b"
```

An individual page overrides the default through its `<website-metadata>`:

```
#metadata((
  title: "Bootstrap confidence intervals",
  image: "assets/bootstrap-card.png",
))<website-metadata>
```

Social scrapers do not resolve relative URLs, so `image` needs `base-url` to be set: without one, no `og:image` or `twitter:image` is emitted. Absolute URLs (`https://cdn.example.com/card.png`) work with or without `base-url`. When an image is available the card is a `summary_large_image`; otherwise it is a plain `summary`.

Redirects

Static hosts serve no redirect rules of their own, so moving a page normally breaks every link that pointed at its old address. List the old routes in a page's `redirect-from` and *Calepin* writes a small stub at each one:

```
#metadata((
  title: "Bootstrap confidence intervals",
  redirect-from: ("posts/bootstrap-ci", "old/bootstrap/"),
))<website-metadata>
```

Each stub carries `<link rel="canonical">` pointing at the new page (so crawlers transfer the old address's ranking), a `<meta http-equiv="refresh">` that moves browsers immediately, and a plain link for readers without JavaScript. Routes are interpreted from the output root; `posts/bootstrap-ci` becomes `posts/bootstrap-ci.html` and a trailing slash becomes `index.html` in that directory. A single string is accepted in place of a list.

A redirect may not overwrite a rendered page, and two pages may not claim the same old route; either is a build error rather than a silent overwrite.

Theme customization

Use a local theme when you want project CSS or template overrides. Point `theme` at the local theme directory and declare the bundled theme you want to extend in `theme.toml`:

```
# calepin.toml
theme = "themes/my-site"
```

```
# themes/my-site/theme.toml
extends = "academic"
```

Place project CSS in `themes/my-site/css/`. Local theme styles load after the parent theme's shared and local styles, so they can adjust colors, fonts, spacing, or component rules. The same theme directory can override HTML layouts, partials, scripts, and `layouts/pdf.typ`.

See Themes for the stable `--calepin-*` CSS tokens exposed by bundled themes.

Paths inside `.typ` files follow Typst path rules, not `calepin.toml` rules. In website builds, use root-relative Typst paths for shared website assets, such as `#image("/assets/diagram.svg")`; the leading `/` points at the website source directory, so the same source works from pages in subdirectories. Calepin's HTML components convert these paths to page-relative browser URLs, so they also work when the website is hosted below a URL prefix. Raw custom HTML attributes are browser URLs rather than Typst paths, so authors are responsible for making those URLs correct for the page and deployment prefix. Ordinary relative Typst paths remain relative and are not rewritten. If no favicon is set, *Calepin* writes a small default to `asset-dir`/favicon.svg; if no logo is set, bundled themes use `title` as the site name.

Syntax highlighting

Configure syntax colors with full TextMate `.tmTheme` files:

```
highlight-light = "themes/syntax/light.tmTheme"
highlight-dark = "themes/syntax/dark.tmTheme"
```

Paths are resolved relative to `calepin.toml`. Both keys apply to every target, and to single documents as well as websites: `calepin compile paper.typ --config calepin.toml` reads the same two keys.

In HTML, the light theme is used for light browser mode and the dark theme for dark browser mode. Paged output has no such switch, so PDF, SVG, and PNG are painted with the light theme.

Calepin writes the resolved paged palette to `asset-dir/syntax.tmTheme` on every build, so a document can hand the same colors to a package that renders `raw` itself:

```
#set raw(theme: "/.calepin/syntax.tmTheme")
```

Such a package intercepts fenced blocks before *Calepin*'s own rules reach them, so without that line its blocks keep Typst's built-in palette while executed chunks use *Calepin*'s. The file is regenerated from `highlight-light` on every build, so the two stay in step. See Tips & tricks for a worked codly example.

For HTML, *Calepin* keeps syntax tokenization consistent with Typst. Typst still reads Sublime syntax definitions (`.sublime-syntax`) and highlights the code; *Calepin* then maps the generated HTML colors to CSS classes so the final colors can depend on light or dark mode.

The HTML mapping uses the TextMate settings that affect rendered spans: `foreground`, `background`, and `fontStyle` values such as `bold`, `italic`, and `underline`. Use light and dark themes with similar scope rules for the most predictable result. If a scope exists in one theme but not the other, *Calepin* falls back to that theme's global foreground.

Robots.txt

Calepin writes `robots.txt` by default:

```
User-agent: *
Allow: /
Sitemap: https://example.com/sitemap.xml
```

The `Sitemap:` line is included when `base-url` is set. To disable generation:

```
robots = false
```

or:

```
[robots]
enabled = false
```

To override the generated file, create `templates/robots.txt` in the website source directory. The template uses MiniJinja syntax and receives `config` and `sitemap_url`:

```
User-agent: *
Disallow: /drafts/
{% if sitemap_url %}Sitemap: {{ sitemap_url }}{% endif %}
```

Files under `templates/` can be included or extended from the robots template:

```
{% extends "base.txt" %}
{% block body %}
User-agent: *
Allow: /
{% endblock %}
```

llms.txt

Calepin writes `llms.txt` at the root of the built site: a Markdown index that lets language models find the site's pages without crawling the rendered HTML.

```
# Example Site

> Notes and papers.

## Pages

- [Home](https://example.com/): The opening paragraph of the home page.
- [Usage](https://example.com/guide/usage.html): How to use it.
```

The heading and blurb come from `title` and `description`. Each entry uses the page title and its excerpt, so pages get a useful one-line description whether or not you wrote a `summary` (see pages). Links are absolute when `base-url` is set and root-relative otherwise.

To disable generation:

```
llms = false
```

Static files

Use `[static]` for files that should be copied to the built website without being rendered as Typst pages:

```
[static]
include = [
  "assets/**",
  "CNAME",
  "downloads/**",
]
exclude = [
  "assets/drafts/**",
]
```

Each included file keeps its source-relative path in the output directory: `assets/logo.svg` becomes `assets/logo.svg`, `CNAME` becomes `CNAME`, and `downloads/manual.pdf` becomes `downloads/manual.pdf`. `include` entries can be files, directories, or glob patterns. `exclude` patterns are applied after includes. Paths must stay inside the website source directory.